NAME:

Practice Problems for Week 6

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 5/11/26**.

You will be submitting the following files:

- caesar.py

Programming Part 1: Caesar Cipher

A Caesar cipher (or shift cipher) encrypts text by "shifting" each letter a given number of positions in the alphabet. For example, if the number is 3, then `a` would become `d`, `b` would become `e` and so on. At the end of the alphabet, we "loop back" to the beginning, so `z` would become a `c`.

For example, if we encode the message "Caesar Ciphers are cool" with a shift of `7`, it becomes:

```
Jhlzhy Jpwolyz hyl jvvs.
```

If we encode that same message with a shift of `1`, it becomes:

```
Dbftbs Djqifst bsf dppm.
```

If you haven't completed the in-class Caesar cipher assignment (aside from the optional parts), you'll need to do that first.

In this section, you will add onto the program you wrote for the previous parts of the assignment. You should have written the functions encipher and decipher, which encode and decode a given string by shifting all letters by a given number of positions in the alphabet.

1. Open the file `caesar.py` which contains your solutions to the last assignment. Add all of your new code to this file. Your task is to write a function called `break_code` that decodes an encrypted message without knowing by how much the letters were shifted.

To do this your function will have to create all possible decodings, compare them and only keep the best one around, which gets returned in the end. For example:

```
>>> break_code('Bzdrzq bhogdq? H oqdedq Bzdrzq rzkzc.')
'Caesar cipher? I prefer Caesar salad.'
```

2. One strategy for determining which decoding is the best is to determine the probability of each candidate decoding by multiplying the letter probabilities for each letter in the decoding and to then choose the candidate decoding with the highest probability. But you are free to use a different strategy if you want to. Be sure to add a good comment to your decoding function explaining what strategy you used.

If you want to use the strategy based on letter probabilities, you can download a file called `letter_probabilities.py`. It provides a function that takes a letter and returns the probability of that letter in English text. Note that the implementation of that function uses dictionaries, which we have not yet discussed in class. However, you do not need to understand how this function is implemented in order to use it. All you need to know is that it is a function called `letter_prob` which takes one parameter (a character) and returns a probability. If the given character is a character from the English language, the function returns the probability of that letter. For all other characters it just returns `1`. To be able to use the function in your program, save the file `letter_probabilities.py` in the same folder as your file `caesar.py` and import it: `from letter_probabilities import letter_prob`

You don't have to and shouldn't modify the file `letter_probabilities.py`.

3. Note: some strings have more than one English deciphering and it is difficult to handle short strings correctly. Thus, your codebreaking function does not have to be perfect. But it should work for longer stretches of English text, e.g., sentences of 8+ words.

Written Questions

1. Convert the following numbers in binary representation to decimal representation and vice versa.

`1011` ..... 34 .....

`11010` ..... 16 .....

`01001` ..... 354 .....

2. Translate the following binary addition problems into decimal representation. (Note the number of underscores is for spacing, it does not indicate the length of the answer.)

binary	decimal	binary	decimal	binary	decimal
`110`		`0110`		`110`	
	`_______`		`_______`		`_______`
`+ 1`	`+______`	`+100`	`+______`	`+10`	`+______`
`111`		`1010`		`1000`	

3. Solve the following binary addition problems. That is, give answers (in binary). (Note the number of underscores is for spacing, it does not indicate the length of the answer.)

`101`	`101`	`0101`	`0101`	`0100`	`010110001`
`+10`	`+01`	`+101`	`+011`	`+111`	`+11000111`
`_____`	`_____`	`_____`	`_____`	`_____`	`_________`

4. A strategy often used when teaching addition in elementary school involves "carrying" the 10 when adding decimal numbers.

For example, applying this strategy to 387 + 595 would look as follows:

```
 11
 387
+595
 982
```

Describe (In English) a similar strategy for adding binary numbers (i.e. give rules to determine which bits should be 1 and which should be 0 to represent the result). This strategy should not convert the binary numbers to decimal numbers. It should just talk about flipping "switches" or binary values.

……

……

……

……

……

……

……

5. Consider the following function definition when answering the questions below:

```
def not_a_good_function_name(alist):
    not_a_good_variable_name = 0
    for item in alist:
        not_a_good_variable_name+= item
    return not_a_good_variable_name
```

5.1 What will get printed to the screen when the function given above is called as follows: `print(not_a_good_function_name([6, 7, 2, 10]))`

……

……

5.2 In general words, what seems to be the purpose of this function?

……

……

……

5.3 Suggest better names for the less than fantastic function and variable names used in this definition.

`not_a_good_function_name` ……

`not_a_good_variable_name` ……